

# Skribble Clone — MERN Roadmap

Real-time multiplayer drawing & guessing game | Sanam Rai | Tech: React, Node.js, Express, Socket.IO, MongoDB, Tailwind CSS

## Suggested Timeline Overview

| Phase | Focus                             | Est. Time |
|-------|-----------------------------------|-----------|
| 0     | Planning & Setup                  | 1-2 days  |
| 1     | Backend Foundation                | 2-3 days  |
| 2     | Frontend Foundation               | 2-3 days  |
| 3     | Rooms & Lobby System              | 2-3 days  |
| 4     | Real-Time Drawing Canvas          | 3-4 days  |
| 5     | Game Logic (Turns, Rounds, Words) | 3-4 days  |
| 6     | Chat & Guessing System            | 2 days    |
| 7     | Scoring & Leaderboard             | 1-2 days  |
| 8     | Polish, UX & Sound                | 2-3 days  |
| 9     | Testing & Bug Fixing              | 2-3 days  |
| 10    | Deployment                        | 1-2 days  |
| 11    | Portfolio & Resume Integration    | 1 day     |

Total estimate: roughly 3-4 weeks part-time. Adjust to your own pace — finishing beats rushing.

## Phase 0: Planning & Setup

Goal: Lock down scope, tools, and repo structure before writing feature code.

■ Do NOT skip the scope doc. Most solo devs die here by adding features mid-build before v1 even works.

### Build order (do these in sequence):

#### 1. Write a one-paragraph scope doc

→ What's in v1 vs cut. Cut list: no private messaging, no custom word packs, no mobile app (responsive web only), English only, no persistent accounts (guest name only).

#### 2. Create the GitHub repo

→ Monorepo with client/ and server/ folders is simpler solo than two repos — one clone, one place to track issues.

#### 3. Set up .gitignore

→ Add node\_modules/, .env, dist/, build/ before your first commit, not after.

#### 4. Write the README skeleton now

→ Just headers: What it is, Stack, Setup, Live Demo. Fill in as you go, not all at the end.

## 5. Build the word list (wordBank.js)

→ 100-150 words across 2-3 categories is enough for v1. A plain JS array is fine — no DB collection needed yet.

## 6. Sketch your 5 core screens on paper

→ Home, Lobby, Game (drawer view), Game (guesser view), Results. Forces you to think through state needs before writing Context.

## 7. Decide your Socket.IO room strategy

→ One Socket.IO room per game room code. Confirm you understand `io.to(roomCode).emit()` before Phase 1.

## Why this phase matters

- Every hour spent planning saves 3+ hours of rework later — same principle as catching bugs early in QA.

# Phase 1: Backend Foundation

Goal: Get Express + MongoDB + Socket.IO running together with a clean base to build on.

■ Get a bare 'server is running + DB connected + socket handshake works' checkpoint BEFORE writing any game logic. Confirm each piece independently — debugging three new things at once (Express + Mongo + Socket.IO) is painful.

## Build order (do these in sequence):

### 1. npm init + install core packages

→ `express`, `mongoose`, `socket.io`, `dotenv`, `cors`, `morgan`, `jsonwebtoken`, `bcrypt`. Dev-only: `nodemon`, `eslint`.

### 2. Split `app.js` from `server.js`

→ `app.js` = Express app + middleware only (testable without a live server). `server.js` = creates the HTTP server, attaches Socket.IO to it, calls `app.listen()`. This split matters later for writing integration tests.

### 3. Connect MongoDB via `config/db.js`

→ Confirm with a `console.log('MongoDB connected')` — don't move on until you see it. This is your first checkpoint.

### 4. Set up `.env` / `.env.example`

→ `PORT`, `MONGO_URI`, `JWT_SECRET`, `CLIENT_URL`. Never commit `.env` — only `.env.example` with blank/placeholder values.

### 5. Add core middleware

→ `CORS` (scoped to `CLIENT_URL` only, not `*`), `Morgan` for request logs, a centralized error handler, and a basic rate limiter. Reuse your Krishi Bazar setup here almost as-is.

### 6. Attach Socket.IO to the HTTP server in `server.js`

→ Test it immediately: `log 'a user connected' on io.on('connection')`. Open your frontend (even a blank Vite app) and confirm the handshake succeeds before building any real features — this is checkpoint 2.

### 7. Build guest-session auth

→ No full account system needed for v1. Player submits a name → `POST /api/auth/guest` → server issues a short-lived JWT with a random `guestId` + name embedded. This JWT is what both REST calls and the socket handshake will use.

### 8. Build `socketAuth` middleware

→ Verify the JWT during the Socket.IO handshake (`io.use()`) so unauthenticated connections are rejected before they can join any room.

### 9. Build your 3 core models

→ `Room` (`roomCode`, `hostId`, `players[]`, `status`, `settings`), `Game` (`roomId`, `rounds[]`, `startedAt`, `endedAt`), `Round` (`word`, `drawerId`, `guesses[]`, `correctGuessers[]`, `startTime`) — can embed `Round` inside `Game` to start, split out later only if

needed.

## Phase 2: Frontend Foundation

*Goal: Scaffold the React app, routing, and shared state before touching game features.*

■ Get the SocketContext talking to your Phase 1 backend before building ANY UI. A blank page that successfully connects/disconnects is the checkpoint — don't build the lobby screen until this works.

**Build order (do these in sequence):**

### 1. Scaffold with Vite + React

→ `npm create vite@latest client -- --template react`. Faster dev server than CRA, and it's what you'll see in most real MERN jobs now.

### 2. Install deps + configure Tailwind

→ `react-router-dom`, `axios`, `socket.io-client`, `tailwindcss`. Run `npx tailwindcss init -p`, configure content paths in `tailwind.config.js`.

### 3. Set up router.jsx

→ Routes: `/ (home)`, `/lobby/:roomCode`, `/game/:roomCode`, `/results/:roomCode`. Stub each page with just a heading for now — routing should work before any real content exists.

### 4. Build SocketContext first, before AuthContext or GameContext

→ One `socket.io-client` instance created once and provided via Context — never call `io()` inside individual components, or you'll get duplicate connections. Test: `connect`, `console.log` the socket id, `disconnect` on `unmount`.

### 5. Build AuthContext

→ Store guest name + JWT (mirrors your Krishi Bazar AuthContext). Persist to `localStorage` so refreshing mid-game doesn't kick the player out.

### 6. Build GameContext

→ Holds room state: `players[]`, `currentDrawer`, `currentRound`, `scores`, `timeRemaining`, `myRole` (`'drawer'` | `'guesser'`). This will be the most-used context in the whole app — get its shape right now, since every game component reads from it.

### 7. Build api.js axios instance

→ `baseUrl` from `import.meta.env.VITE_API_URL`, with an interceptor attaching the JWT — same pattern you already built for Krishi Bazar.

## Phase 3: Rooms & Lobby System

*Goal: Players can create a room, share a code, and see who's joined before the game starts.*

■ This is your first full vertical slice — REST + Socket.IO + MongoDB + React all working together on one feature. Get 2 browser tabs joining the same room and seeing each other before moving to Phase 4. This checkpoint proves your whole stack is wired correctly.

**Build order (do these in sequence):**

### 1. Build POST /api/rooms (REST)

→ Creates a Room doc, generates a short unique code (e.g. 6 chars via `generateRoomCode.js`), returns it. This is a plain REST call, not a socket event — room creation doesn't need to be real-time.

### 2. Build GET /api/rooms/:code (REST)

→ Validates the room exists and isn't full or already started, before the frontend even attempts a socket join. Fail fast with a clear error message.

### 3. Build RoomManager.js (in-memory)

→ Map of roomCode → { players, hostId, status }. This mirrors the DB but lives in memory for speed — sockets read/write here constantly during a game; you don't want a DB round-trip on every player action.

### 4. Build the join-room socket event

→ Client emits join-room with roomCode + JWT. Server verifies via socketAuth, adds player to both the Room doc and RoomManager, then broadcasts the updated player list to everyone in that room.

### 5. Build leave-room / disconnect handling

→ Socket.IO's built-in 'disconnect' event covers tab closes and crashes, not just intentional leaves. Reassign host if the host disconnects — don't let a room get orphaned.

### 6. Build HomePage forms

→ Create Room (name input, calls POST /api/rooms then navigates to lobby) and Join Room (name + code input, calls GET /api/rooms/:code to validate, then navigates).

### 7. Build LobbyPage

→ Shows the room code (with a copy-to-clipboard button — small detail, big UX win), live player list updating via GameContext, and a host-only 'Start Game' button.

### 8. Handle UI edge cases

→ Room full, room not found, room already started — test each by manually triggering them, not just the happy path.

## Phase 4: Real-Time Drawing Canvas

Goal: The core interaction: one player draws, everyone else sees it live. This is the hardest phase — budget the most time and patience here.

■ Build and test this in ISOLATION first — a throwaway 2-player test room with no game logic at all, just drawing sync. Don't combine it with turn/round logic (Phase 5) until sync itself feels instant and reliable.

### Build order (do these in sequence):

#### 1. Build local-only drawing first (no sockets yet)

→ DrawingCanvas.jsx with mousedown/mousemove/mouseup listeners drawing directly to canvas via CanvasRenderingContext2D. Get this working and feeling good before adding any network code — isolates canvas bugs from sync bugs.

#### 2. Capture stroke points in a clean shape

→ {x, y, color, brushSize, type: 'start'|'move'|'end'}. Draw locally immediately on every event — never wait for a server round-trip to render your OWN strokes, only for other players' strokes.

#### 3. Add touch event support alongside mouse events

→ touchstart/touchmove/touchend. Easy to forget until you test on a phone, and cheap to add now vs. retrofitting later.

#### 4. Wire up stroke-start / stroke-move / stroke-end socket events

→ Drawer emits, server does io.to(roomCode).except(senderSocketId).emit() to relay to everyone else in the room but not echo back to the drawer.

#### 5. Throttle stroke-move emissions

→ Batch/throttle to roughly every 20-30ms (not every single mousemove pixel) — otherwise you'll flood the socket and cause visible lag on slower connections. This is the #1 thing that separates a janky drawing sync from a smooth one.

## 6. Replay incoming strokes on other clients

→ Listener draws received points onto the local canvas using the same drawing function you built in step 1 — reuse the render logic, don't duplicate it.

## 7. Build Toolbar.jsx

→ Color picker, brush size slider, clear-canvas button. Add undo only if time allows — it's a nice-to-have, not core.

## 8. Wire clear-canvas as a broadcast socket event

→ Same room-broadcast pattern as strokes.

## 9. Lock canvas interaction based on role

→ Check myRole from GameContext — only the current drawer's mousedown handler should be active; guessers' canvas is view-only.

## 10. Test with 2+ real browser tabs side by side

→ Not just localhost — test with your phone on the same wifi hitting your dev machine's local IP, to catch latency issues early.

# Phase 5: Game Logic — Turns, Rounds & Words

Goal: Server-authoritative game flow: who draws, what word, how long, what happens next.

■ Everything here must be decided by the SERVER, never the client. If the client decides 'time's up' or 'that guess is correct,' players can cheat by editing browser JS. This is the same mindset as your JWT middleware — never trust the client.

### Build order (do these in sequence):

#### 1. Build GameManager.js in-memory state

→ roomCode → { currentDrawerIndex, currentWord, roundNumber, status, roundStartTime }. Lives alongside RoomManager from Phase 3.

#### 2. Build turn rotation logic

→ Simple index cycling through players[] —  $currentDrawerIndex = (currentDrawerIndex + 1) \% players.length$ . Handle the edge case of a player leaving mid-rotation without breaking the cycle.

#### 3. Build word selection flow

→ Server picks 3 random words from wordBank.js, emits word-choices to the drawer only (`io.to(drawerSocketId).emit()`), drawer picks one via a select-word socket event, server stores it as currentWord.

#### 4. Build the server-side round timer

→ Store roundStartTime as a timestamp, not a countdown variable — client calculates remaining = roundDuration - (Date.now() - roundStartTime) for display, but the SERVER independently runs its own setTimeout to actually end the round. Never let the client's timer trigger game-ending logic.

#### 5. Build round-start broadcast

→ Sends masked word ('\_ \_ \_ \_ \_') to guessers, full word to drawer only — two different emits, not one broadcast with conditional rendering (don't leak the word to guessers' network tab).

#### 6. Build round-end logic

→ Triggered by either timer expiry OR all-guessers-correct (checked in Phase 6). Broadcasts the real word to everyone, freezes the canvas, shows round results briefly.

#### 7. Build next-round transition

→ Advance drawer index, reset canvas (emit clear-canvas), pick new word, short delay/countdown before round-start fires again.

## 8. Frontend: WordSelectModal.jsx

→ Shown only to the drawer, 3 buttons. Auto-picks a random one if the drawer doesn't choose within a few seconds, so the game doesn't stall.

## 9. Frontend: WordDisplay.jsx + RoundTimer.jsx

→ WordDisplay renders blanks for guessers / full word for drawer. RoundTimer displays the countdown calculated from the server timestamp — resync periodically, don't just start a local setInterval and trust it (client tabs can throttle timers when backgrounded).

## 10. Frontend: role-based UI switching

→ Drawer sees Toolbar + word choice; guessers see GuessBox instead. Drive this off myRole in GameContext, not by guessing based on component mount order.

# Phase 6: Chat & Guessing System

Goal: Reuse your YakTalk real-time messaging pattern, adapted for guess-checking.

■ This should be the fastest phase — you've already solved real-time messaging in YakTalk. The only new logic is the word-matching check on the server.

### Build order (do these in sequence):

#### 1. Build the send-guess socket event

→ Server checks the guess against currentWord — case-insensitive, trimmed of whitespace. Do this check server-side only, since the drawer's word is never sent to guessers' clients.

#### 2. Handle a correct guess

→ Mark that player 'guessed' for this round in GameManager, broadcast a highlighted system message ('Player X guessed the word!') without revealing the word itself, and block that player from submitting further guesses this round (both server-side check and UI disable).

#### 3. Handle an incorrect guess

→ Broadcast it as a normal chat message to the whole room — this is literally your YakTalk send-message logic, unchanged.

#### 4. Block the drawer from guessing

→ Check role server-side before processing any send-guess event — don't rely on hiding the GuessBox in the UI alone, since a client could still emit the event manually.

#### 5. Build ChatPanel.jsx + ChatMessage.jsx

→ Three visual message types: normal guess, correct-guess (highlighted/colored), system message (round start/end announcements). Reuse your YakTalk chat components, just extend the styling.

#### 6. Wire auto-end-round when everyone's guessed

→ Check in the correct-guess handler: if guessedCount === (players.length - 1) [excluding drawer], trigger round-end early instead of waiting for the timer.

# Phase 7: Scoring & Leaderboard

Goal: Turn correct guesses into points, and persist results.

### Build order (do these in sequence):

#### 1. Build scoring.js with a simple time-based formula

→ e.g.  $\text{points} = \max(100 - \text{secondsElapsed} * 5, 10)$  — faster guess = more points, with a floor so late guesses still earn something. Keep the formula simple for v1; you can tune it later.

## 2. Award the drawer bonus points

→ Based on how many players guessed correctly this round — incentivizes drawing clearly, not just participating.

## 3. Update in-memory scores in GameManager on each correct guess

→ Then broadcast a score-update event so ScoreBoard.jsx updates live, not just at round end.

## 4. Build ScoreBoard.jsx

→ Live sidebar, sorted descending by score, small animation on score change if you have time (nice polish, not required for v1).

## 5. Persist final results to MongoDB on game end

→ Save the completed Game doc with all Round data and final scores — this is what makes a 'past games' or leaderboard-across-games feature possible later, even if you don't build that UI in v1.

## 6. Build ResultsPage.jsx

→ Final standings, winner highlight, 'Play Again' (reuses the same room) and 'Back to Home' buttons.

# Phase 8: Polish, UX & Sound

Goal: The layer that makes it feel like a real game instead of a working prototype.

■ Only start this once Phases 1-7 work end-to-end. Polishing a broken game wastes time — polishing a working one is what makes it demo well.

## Build order (do these in sequence):

### 1. Add sound effects

→ Correct guess, round-end, timer tick (final 5 seconds). Keep them short and optional — add a mute toggle, since auto-playing sound is an easy way to annoy a recruiter clicking around your portfolio.

### 2. Add transitions between lobby → game → results

→ Simple CSS transitions/fades are enough — don't reach for a heavy animation library for this.

### 3. Add loading/connecting states

→ Reuse your Krishi Bazar shimmer pattern for the moment between clicking 'join' and the socket confirming connection.

### 4. Verify mobile/touch support end-to-end

→ You added touch listeners in Phase 4 — now actually test drawing and guessing on a real phone, not just Chrome DevTools device mode.

### 5. Responsive layout pass

→ Canvas + sidebar need to stack sensibly on small screens — this is often where a rushed layout looks worst.

### 6. Handle empty/edge states explicitly

→ Room full, host disconnected mid-game, only 1 player left, word list exhausted. Each of these needs an actual UI message, not a silent freeze.

### 7. Add per-player avatar/color assignment

→ Simple — assign from a fixed color palette by join order. Makes the player list and score board easier to scan at a glance.

## Phase 9: Testing & Bug Fixing

*Goal: Catch the real-time edge cases before they catch you in a demo.*

■ Real-time apps fail in ways CRUD apps don't — race conditions, disconnects, and timing bugs. This is exactly where your QA background gives you an edge over other junior devs, so don't rush this phase.

### Build order (do these in sequence):

#### 1. Full end-to-end manual test with 3+ tabs/devices

→ Play a complete game start to finish. Write down every bug you hit as you go instead of trying to remember them after.

#### 2. Test disconnects mid-round

→ Host leaves, drawer leaves, a guesser leaves — each should degrade gracefully (reassign host, skip drawer's turn, just remove the guesser) rather than freezing the game for everyone else.

#### 3. Test simultaneous correct guesses

→ Two players submitting the correct guess in the same instant — confirm your server-side check handles this without double-counting or crashing (classic race condition).

#### 4. Write a handful of backend unit tests

→ `scoring.js` and `maskWord.js` are the easiest pure functions to test — use Jest (Node's standard, equivalent to your Pytest experience).

#### 5. Test under throttled network conditions

→ Chrome DevTools → Network → Slow 3G. This is where stroke-lag and timer-desync bugs actually show up — they're invisible on localhost.

#### 6. Check for memory leaks in RoomManager/GameManager

→ Confirm rooms actually get deleted from the in-memory maps once everyone leaves — otherwise your server's memory grows every game played and never releases it.

## Phase 10: Deployment

*Goal: Ship it the same way you shipped your other projects — Render + Vercel.*

### Build order (do these in sequence):

#### 1. Deploy backend to Render as a Web Service

→ Same as your other projects — Render handles Socket.IO fine on the Web Service tier (not Static Site).

#### 2. Deploy frontend to Vercel

→ Set env vars (`VITE_API_URL`, `VITE_SOCKET_URL`) pointing at your Render backend URL before building.

#### 3. Configure CORS to allow your exact Vercel domain

→ Not a wildcard — you already handled this pattern in Krishi Bazar.

#### 4. Confirm Socket.IO connects over WSS in production

→ Production uses `wss://` not `ws://` — test this specifically, since it's a common silent-failure point that works fine on localhost and breaks only after deploy.

#### 5. Set up MongoDB Atlas production DB + IP allowlist

→ Allow Render's IP range (or `0.0.0.0/0` if you're on a dynamic-IP Render plan, understanding the tradeoff).

#### 6. Do a full live playtest on the deployed URL with real friends

→ Different networks, different devices — this is your final real-world check before calling v1 done.

## Phase 11: Portfolio & Resume Integration

*Goal: Make sure the work actually gets seen and understood by whoever's hiring.*

■ A great project with a bad README or no demo video gets skipped in a 30-second resume scan. This phase is not optional — it's where the work you did actually gets credit.

### **Build order (do these in sequence):**

#### **1. Write a strong README**

→ What it is, tech stack, an architecture diagram (even a simple one), how to run it locally, and the live demo link at the top — not buried at the bottom.

#### **2. Add the project to sanam-rai.com.np**

→ With screenshots and a gameplay GIF — this project is genuinely fun to watch, so lead with the visual, not just a text description.

#### **3. Record a short gameplay GIF or Loom video**

→ 10-20 seconds of two players drawing and guessing live is worth more than any paragraph of description — this is the project's biggest selling point and it's invisible in plain text.

#### **4. Add an impact-style bullet to your CV**

→ Match the style of your existing project bullets — lead with what it does, then the technically hard part (e.g. 'server-authoritative round timing to prevent client-side cheating').

#### **5. Prepare your 30-second interview explanation**

→ What problem it solves, and what was technically hardest — for this project, that's almost certainly stroke-sync throttling or server-authoritative timing. Practice saying this out loud once or twice.